

Lab 2D: Package and Ship - Plugins & Marketplaces

Duration: 35 min

After: Plugins & Marketplaces lecture (Day 2, Block 4)

Prerequisites: Labs 2A-2C complete; `git config user.name` and `git config user.email` return configured values so `plugin init` can populate author metadata for strict validation (skills and hooks in `.claude/`)

Objective

Inspect a marketplace entry before installation, scaffold a skills-directory plugin with `claude plugin init`, validate it strictly, publish the proven package through a local marketplace, and install it back at an intentional scope.

Deliverable (toolkit chain 8 of 12)

A validated `dashboard-toolkit` plugin containing the model-invoked review skill, the manual-only ship checklist, and two hooks, plus a local marketplace that can distribute it.

START MARKER: `/clear` done; open the `/` menu, select **Skills**, and confirm

`dashboard-code-review` and `ship-checklist`; both hooks and the Lab 2B regression matrix passed.

Exercise 1: Inspect Before Installing (6 min)

In the Claude Code VS Code panel, type `/` and select **Manage plugins** from the menu. Do not type `/plugin`; that slash command belongs to the terminal CLI interface and is not available in the VS Code extension panel.

Choose a plugin from `claude-plugins-official`, open its detail view, and inspect the publisher plus every contributed component. The official marketplace is a catalog; entries can have different publishers and maintenance models.

Use this review checklist:

1. Publisher, source repository, release activity
2. Skills, commands, agents, hooks, scripts, MCP/LSP servers, dependencies
3. Files, commands, credentials, and user privileges it can access
4. Network destinations, model/provider changes, and stored state
5. Version/update policy plus disable, uninstall, and rollback path

Record an **APPROVE**, **REJECT**, or **APPROVE WITH CONDITIONS** decision with one sentence of evidence.

Plugins are executable dependencies. Marketplace placement is not a security boundary.

Exercise 2: Scaffold the Plugin (6 min)

Open the VS Code integrated terminal with **Terminal > New Terminal**. You do not need to close VS Code.

Run this exact one-line command in PowerShell or Bash:

```
claude plugin init dashboard-toolkit --with skills hooks
```

The command creates `~/.claude/skills/dashboard-toolkit/` and loads it as

`dashboard-toolkit@skills-dir` on the next session. It does not scaffold in the current directory. You will set the final manifest description when you package the toolkit in Exercise 3.

After the command succeeds, start a new Claude Code extension session in the same project before checking for the plugin.

Inspect and validate:

```
claude plugin validate "$HOME/.claude/skills/dashboard-toolkit" --strict
claude plugin list
```

Expected output marker: strict validation succeeds and the plugin appears from the `skills-dir` source.

Exercise 3: Package the Toolkit (8 min)

Start Claude in `business-dashboard` and run:

```
Update $HOME/.claude/skills/dashboard-toolkit so it contains:
1. My .claude/skills/dashboard-code-review/ skill as skills/dashboard-code-review/
2. My .claude/skills/ship-checklist/ manual-only skill as skills/ship-checklist/
3. My post-edit-format.sh and pre-write-check.sh scripts under hooks/
4. hooks/hooks.json declaring the PostToolUse Write|Edit formatter and the final PreToolUse Write|Edit secret blocker using ${CLAUDE_PLUGIN_ROOT}-relative paths; preserve checks for both tool_input.content and tool_input.new_string
5. A manifest description that names the skills and hooks
Remove placeholder example components that the scaffold created.
Do not add an MCP server, external network dependency, credential, or provider override.
```

Validate again:

```
claude plugin validate "$HOME/.claude/skills/dashboard-toolkit" --strict
claude plugin details dashboard-toolkit@skills-dir
```

Expected output marker: validation passes and the component inventory matches the intended package. Review the inventory before continuing.

Exercise 4: Publish Through a Local Marketplace (10 min)

Create the marketplace **outside the project folder, as a sibling of it**. If your project is

`C:\AIA-Code\Labs\business-dashboard`, the marketplace root is

`C:\AIA-Code\Labs\team-marketplace`. Do not place `team-marketplace` inside `business-dashboard`.

The parent folder should look like this:

```
Labs/business-dashboard/ [existing project]
Labs/team-marketplace/ [new sibling]
team-marketplace/.claude-plugin/marketplace.json
team-marketplace/plugins/dashboard-toolkit/ contains:
- .claude-plugin/plugin.json
- skills/
- hooks/
```

If your project folder has a different name, such as `claudecode-lab1b`, use that existing name in place of `business-dashboard`; the sibling relationship is the same.

Copy the validated plugin from your personal skills directory into the marketplace package. From the `business-dashboard` project root, run these commands in the VS Code **PowerShell** terminal:

```
$src = "$HOME\.claude\skills\dashboard-toolkit"
$dst = "..\team-marketplace\plugins\dashboard-toolkit"
New-Item -ItemType Directory -Force -Path $dst | Out-Null
Copy-Item -Path "$src\*" -Destination $dst -Recurse -Force
Get-ChildItem -Path $dst -Force
```

Expected output marker: the destination contains `.claude-plugin`, `skills`, and `hooks`. The source under `$HOME\.claude\skills\dashboard-toolkit` remains intact.

Before installing the marketplace copy, retire the enabled scaffold copy so two plugins with the same name cannot compete:

```
claude plugin disable dashboard-toolkit@skills-dir
```

The project-settings hooks from Lab 2B remain active alongside the plugin hooks until you remove them. For this lab, keep them and expect both registrations to run; do not mistake duplicate enforcement for a plugin failure.

Create the marketplace manifest by sending this prompt in the Claude Code VS Code panel. Approve access only to the named sibling path if the extension asks:

Create `../team-marketplace/.claude-plugin/marketplace.json` with exactly this JSON. Create the `.claude-plugin` parent folder if needed. Do not modify any other file. After writing it, parse the saved file as JSON and report the path.

```
{
  "name": "team-marketplace",
  "description": "Locally validated Day 2 training marketplace for team extensions",
  "owner": { "name": "Training Team" },
  "plugins": [
    {
      "name": "dashboard-toolkit",
      "source": "../plugins/dashboard-toolkit",
      "description": "Review skills and deterministic enforcement hooks"
    }
  ]
}
```

Expected output marker: Claude reports that

`../team-marketplace/.claude-plugin/marketplace.json` parses as valid JSON. Open the file and verify its plugin source is `../plugins/dashboard-toolkit` before continuing.

From the `business-dashboard` VS Code integrated terminal, run:

```
claude plugin marketplace add ../team-marketplace
claude plugin install dashboard-toolkit@team-marketplace --scope local
```

Choose **local scope** for the classroom smoke test unless the instructor explicitly asks you to share the project configuration.

Then verify:

```

claude plugin list
claude plugin details dashboard-toolkit@team-marketplace

```

If the install summary requests activation, send this in the Claude Code VS Code panel. Unlike `/plugin`, which is not a VS Code panel command, `/reload-plugins` is a panel command. If it is not offered, restart the Claude Code extension session instead:

```
/reload-plugins
```

Test a natural-language review prompt and confirm the plugin's `dashboard-code-review` skill produces its RED/YELLOW/BLUE contract. Then explicitly run `/dashboard-toolkit:ship-checklist`. Verify the secret blocker separately against both Write content and Edit new_string using the exact unsafe fixture from Lab 2B.

Confirm `process.env.API_KEY` remains allowed.

Exercise 5: Prove Reversibility (5 min)

```

claude plugin disable dashboard-toolkit@team-marketplace --scope local
claude plugin enable dashboard-toolkit@team-marketplace --scope local
claude plugin uninstall dashboard-toolkit@team-marketplace --scope local

```

Reinstall at local scope after proving the lifecycle:

```
claude plugin install dashboard-toolkit@team-marketplace --scope local
```

Record which settings file changed for the selected scope.

VS Code Acceptance Gate and Process Receipt

Before commit, capture the same supervised workflow used across Day 2:

1. **Orient:** select the target range in the editor and use `Alt+K` or an explicit `@file#line-range`; explain unfamiliar code before editing.
2. **Plan:** review the bounded plan in the Claude Code panel; add an inline comment when the plan misses a constraint.
3. **Approve and implement:** approve only the requested scope and exact permissions.
4. **Inspect diff:** review every changed file in VS Code Source Control and reject unexpected files or authority changes.
5. **Verify:** clear the Problems panel for the touched scope and run the lab's deterministic tests or fixtures in the integrated terminal.
6. **Decide:** record the request, approved plan, changed files, verification evidence, and the human commit/install decision.

A model summary or grader result supports this receipt; it does not replace the diff review.

FINISH MARKER - Success Checklist

- [] Inspected exact publisher, components, authority, data flow, and lifecycle before installation
- [] Recorded an evidence-based trust decision
- [] `claude plugin init dashboard-toolkit --with skills hooks` created a skills-directory plugin

- [] `claude plugin validate ... --strict` passed
- [] Marketplace uses `./plugins/dashboard-toolkit` inside its own repository
- [] Plugin installed at intentional scope and activated successfully
- [] Plugin review skill and namespaced ship checklist work; final Write|Edit secret blocker rejects both Write and Edit fixtures while allowing the environment-variable fixture
- [] Disable, enable, uninstall, and reinstall paths were proven

If Stuck

Problem	Recovery
Scaffold not in current folder	Expected: it is under <code>~/.claude/skills/dashboard-toolkit/</code>
Strict validation fails	Fix every schema error or warning before publication
Marketplace source not found	Keep plugin under <code>team-marketplace/plugins/</code> and use a marketplace-relative source
Plugin installed but inactive	Run the panel command <code>/reload-plugins</code> when offered; otherwise restart the Claude Code extension session. Do not type <code>/plugin</code> in the panel
Old component remains	Remove the installed copy, update the marketplace, and reinstall at the intended scope
Hook path breaks after install	Use <code>\${CLAUDE_PLUGIN_ROOT}</code> in plugin hook paths

Minimum path if behind: finish Exercises 2-3, copy the validated plugin into the sibling marketplace, create the provided marketplace manifest, run strict validation locally, install at local scope, invoke one packaged skill, and prove the final Write|Edit blocker. Lifecycle disable/uninstall is stretch if time remains.

Debrief Questions

1. Why is a marketplace a distribution mechanism rather than a security boundary?
2. Which component in your plugin carries the most authority?
3. What evidence would your organization require before moving this plugin into an approved internal marketplace?

Optional Challenge: Produce an Installation Approval Record (8-10 min)

Finish the core lab first. Create `team-marketplace/INSTALL_APPROVAL.md` from the exact marketplace copy, not from memory. Record:

1. Marketplace and plugin name, publisher/owner, source path, and installed scope.
2. Every packaged skill and hook.
3. Allowed and disallowed tools for each skill.
4. Hook events, matchers, commands, and whether each hook advises or blocks.
5. External network, credential, MCP/LSP, provider, and dependency findings.
6. Strict-validation result plus the disable, uninstall, reinstall, and rollback commands you proved.
7. One verdict: `APPROVE`, `REJECT`, or `APPROVE WITH CONDITIONS`, with evidence.

Compare the record with `claude plugin details dashboard-toolkit@team-marketplace`. Correct any mismatch

in the record or package, then rerun strict validation if the package changed.

Challenge done when: the approval record matches the installed component inventory and another administrator could understand the plugin's authority and reverse the installation without guessing.